

Pedestrian Analysis — Backend Developer & Intern Guide

Welcome to the **ANPR & Pedestrian Analytics Backend**! This document provides a complete technical deep-dive into how pedestrian detection, tracking, rider filtering, region-of-interest (ROI) spatial validation, crowd alerting, and live streaming operate across the codebase.

Table of Contents

1. [Executive Architecture Overview](#)
 2. [End-to-End Execution Flowchart](#)
 3. [Core Subsystems & Module Map](#)
 4. [Deep-Dive: Pedestrian AI Detection Engine](#)
 - [4.1 Model & Target Classes](#)
 - [4.2 Rider Filtering Algorithm](#)
 - [4.3 Polygon ROI Spatial Testing](#)
 - [4.4 Frame Annotation & Visual Output](#)
 5. [Orchestration, Alerting & Cooldown Management](#)
 - [5.1 Pipeline Loop Integration](#)
 - [5.2 Crowd Threshold & Alert Cooldown Logic](#)
 - [5.3 Alert Persistence & Snapshot Storage](#)
 6. [API Specification for Developers](#)
 - [6.1 Video Upload with Pedestrian Config](#)
 - [6.2 Live RTSP Camera Job Creation](#)
 - [6.3 Setting ROI & Triggering Background Pipeline](#)
 - [6.4 Fetching Results & Crowd Alerts](#)
 - [6.5 WebSocket Live Streaming](#)
 7. [Database Schema & Entity Relations](#)
 8. [Notification & Email Reporting](#)
 9. [Intern Onboarding & Debugging Guide](#)
-

1. Executive Architecture Overview

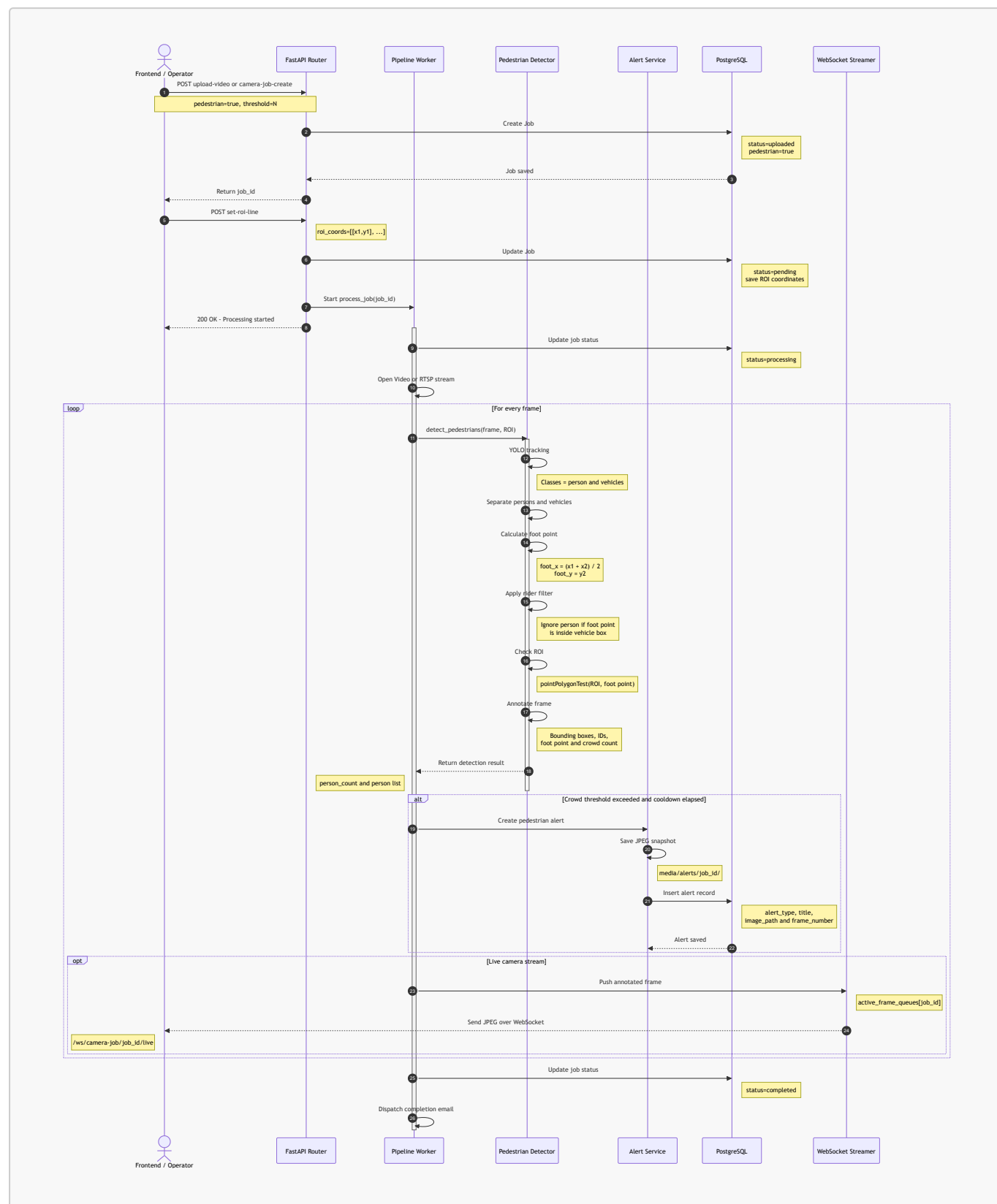
The backend is built with **FastAPI**, **SQLAlchemy** (PostgreSQL/SQLite), **Ultralytics YOLO**, **OpenCV**, and **PaddleOCR**.

While the system handles Automatic Number Plate Recognition (ANPR), the **Pedestrian Analysis Subsystem** is an optional module within the computer vision pipeline. When enabled:

- It tracks human beings in video files or live RTSP streams.
- Filters out individuals riding vehicles (motorcycles, bicycles).
- Tests if remaining pedestrians are inside a user-defined **Region of Interest (ROI)** polygon.
- Monitors crowd counts against a configurable threshold (`pedestrian_threshold`).
- Triggers persistent alerts with frame snapshots and sends summary emails upon job completion.

2. End-to-End Execution Flowchart

The following sequence details how frame data flows from HTTP/RTSP ingest through AI inference to DB storage and WebSocket dispatch:



3. Core Subsystems & Module Map

Module File	Responsibility
app/main.py	FastAPI app initialization, CORS, database table creation, schema auto-migration.
app/api/routes.py	REST API endpoints for job creation, ROI setup, results retrieval, and WebSocket streaming.
app/ai/pedestrian_detector.py	Core pedestrian AI module (YOLO tracking, rider filtering logic, foot-point calculation, ROI testing).
app/ai/pipeline_with_tracking.py	Master frame loop orchestrator combining vehicle detection, plate OCR, and pedestrian alerting.
app/services/alert_service.py	Service layer responsible for deduplicating, capturing snapshot images, and saving DB alert records.
app/services/job_manager.py	Job status manager and background worker entry point (<code>process_job</code>).
app/db/models.py	SQLAlchemy ORM models (<code>Job</code> , <code>Alert</code> , <code>Plate</code> , <code>User</code>).
app/email/notification.py	Prepares statistical metrics (total pedestrian alerts) and sends summary HTML emails.

4. Deep-Dive: Pedestrian AI Detection Engine

Located in `app/ai/pedestrian_detector.py`, the `detect_pedestrians()` function processes raw BGR video frames.

4.1 Model & Target Classes

The detector initializes Ultralytics YOLO with pre-trained or fine-tuned pedestrian weights:

```
model = YOLO("models/pedestrian.pt")

PERSON_CLASS = 0
VEHICLE_CLASSES = [1, 2, 3, 5, 7] # bicycle, car, motorcycle, bus, truck
```

When tracking is invoked:

```
results = model.track(
    frame,
    persist=True,
    classes=[0, 1, 2, 3, 5, 7],
    verbose=False,
)
```

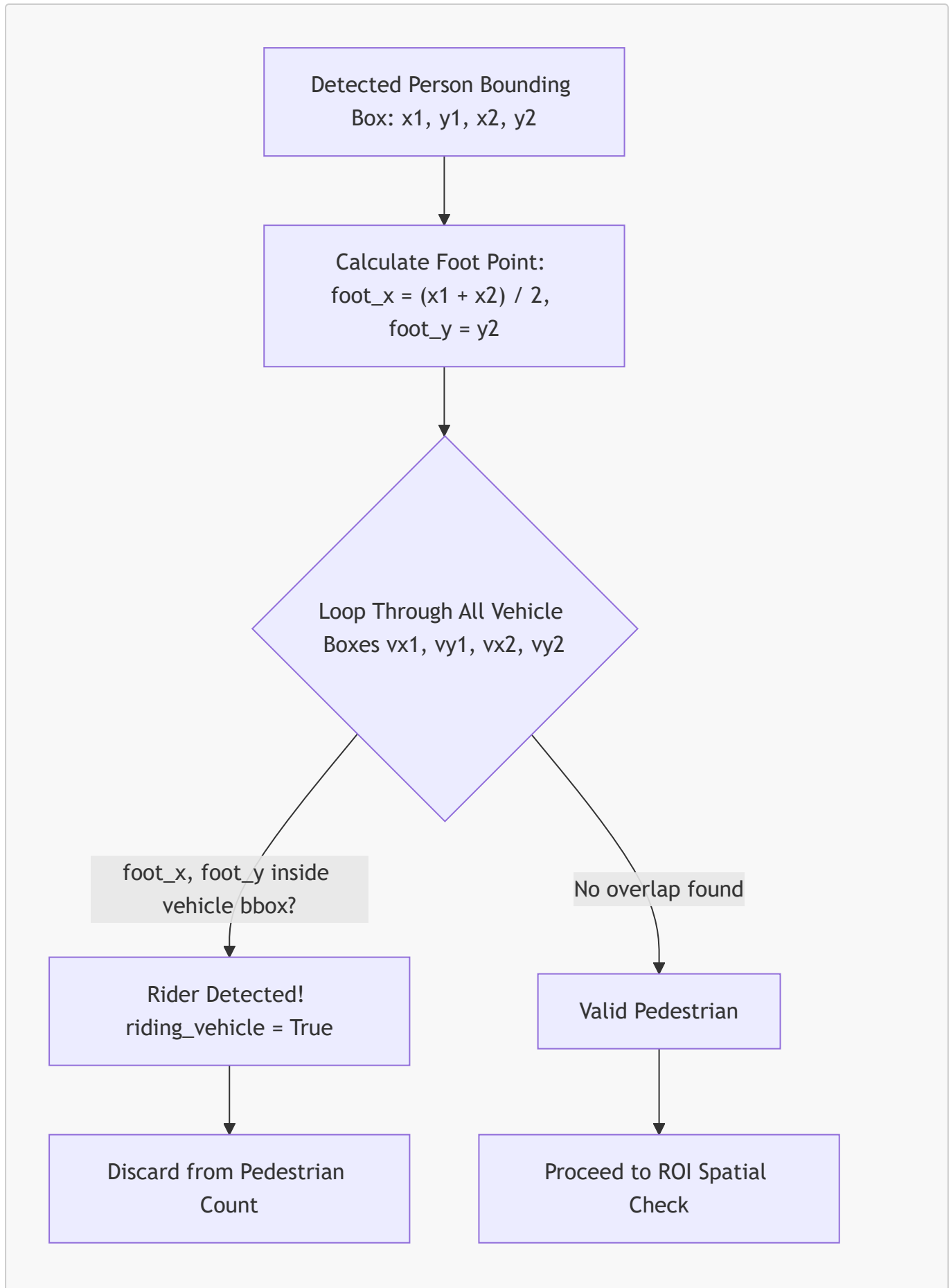
- `persist=True`: Maintains track IDs across consecutive video frames.

- Multi-class tracking ensures both persons and nearby vehicles are identified in a single forward pass.
-

4.2 Rider Filtering Algorithm

A common challenge in computer vision is distinguishing **freely walking pedestrians** from **people riding vehicles** (e.g., bicyclists, motorcyclists). Counting riders as pedestrians leads to false crowd alerts on roadways.

The system resolves this using **Foot-Point Spatial Intersect Testing**:



Code Snippet ([app/ai/pedestrian_detector.py](#)):

```

foot_x = (x1 + x2) // 2
foot_y = y2

riding_vehicle = False
for vx1, vy1, vx2, vy2 in vehicle_boxes:
    if vx1 <= foot_x <= vx2 and vy1 <= foot_y <= vy2:
        riding_vehicle = True
        break

if riding_vehicle:
    continue # Skip rider, do not count as pedestrian

```

4.3 Polygon ROI Spatial Testing

Users specify custom Region-of-Interest (ROI) polygons (e.g., sidewalks, entrances, restricted zones).

The system tests if the pedestrian's **foot point** (`foot_x`, `foot_y`) falls within the polygon using OpenCV's `pointPolygonTest`:

```

inside_roi = True
if roi_polygon is not None:
    inside = cv2.pointPolygonTest(
        roi_polygon,
        (float(foot_x), float(foot_y)),
        False,
    )
    inside_roi = inside >= 0

```

- `inside >= 0`: Point is inside or on the boundary of the ROI polygon.
- `person_count`: Incremented only when `inside_roi` evaluates to `True`.

4.4 Frame Annotation & Visual Output

Every frame processed by `detect_pedestrians()` is visually annotated in real-time:

```

# Bounding Box Color Scheme (BGR)
PERSON_COLOR = (0, 255, 0) # Green for pedestrians inside ROI
OUTSIDE_COLOR = (0, 0, 255) # Red for pedestrians outside ROI
FOOT_COLOR = (255, 255, 0) # Cyan circle at foot point
ROI_COLOR = (255, 0, 255) # Magenta for ROI boundary polygon

```

1. **Bounding Box**: Bounded with `PERSON_COLOR` (Green) if inside ROI, or `OUTSIDE_COLOR` (Red) if outside.
2. **ID Tag**: `ID:{track_id}` displayed above the head (`y1 - 10`).
3. **Foot Circle**: Filled circle drawn at (`foot_x`, `foot_y`).

4. **Crowd HUD:** Top-left text overlay: "Persons Inside ROI : {person_count}".

5. Orchestration, Alerting & Cooldown Management

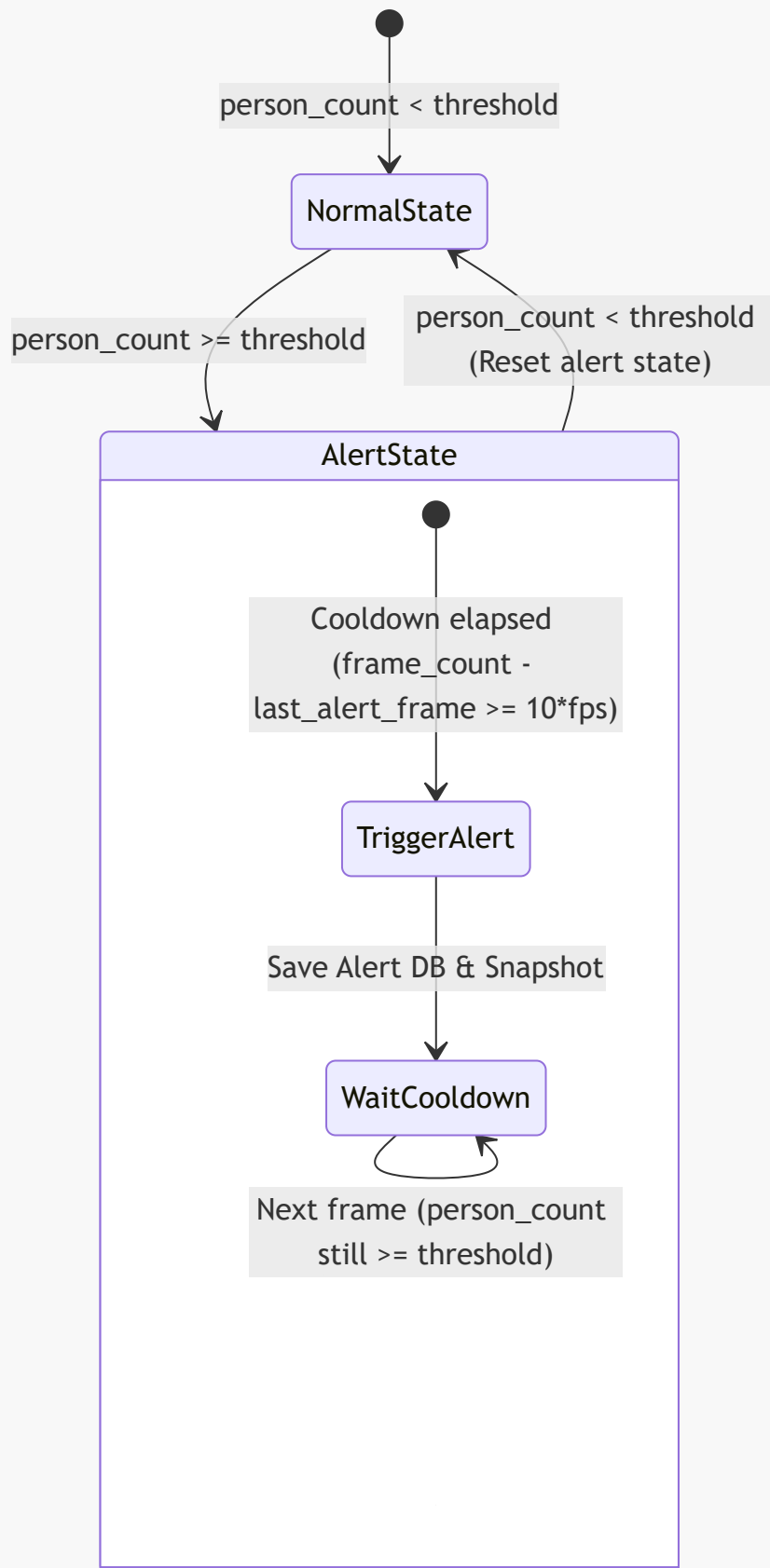
5.1 Pipeline Loop Integration

In `app/ai/pipeline_with_tracking.py`, the `run_pipeline_with_tracking()` loop coordinates pedestrian analysis alongside ANPR vehicle tracking:

```
if pedestrian_enabled:
    pedestrian_result = detect_pedestrians(
        frame=frame,
        display_frame=display_frame,
        roi_polygon=(
            np.array(roi_polygon, dtype=np.int32) if roi_polygon else None
        ),
    )
    person_count = pedestrian_result["person_count"]
    threshold = job.pedestrian_threshold
```

5.2 Crowd Threshold & Alert Cooldown Logic

To prevent flooding the database with dozens of alerts per second while a crowd remains in the frame, an **Alert Cooldown State Machine** is implemented:

**Cooldown Formula:**

```
ALERT_COOLDOWN_FRAMES = int(fps * 10) # Cooldown = 10 seconds of video footage
```



```
if person_count >= threshold:
    can_create_alert = not pedestrian_alert_active or (
        frame_count - last_pedestrian_alert_frame >= ALERT_COOLDOWN_FRAMES
    )

    if can_create_alert:
        create_alert(
            db=db,
            job_id=job.job_id,
            alert_type="pedestrian",
            title="Crowd Threshold Exceeded",
            description=f"{person_count} persons detected inside ROI",
            frame=display_frame,
            frame_number=frame_count,
        )
        pedestrian_alert_active = True
        last_pedestrian_alert_frame = frame_count
    else:
        pedestrian_alert_active = False # Reset state when crowd disperses
```

5.3 Alert Persistence & Snapshot Storage

When `create_alert()` in `app/services/alert_service.py` is invoked:

1. Directory check/creation: `media/alerts/{job_id}/`.
2. Image export: Saves the current annotated frame as `pedestrian_{frame_number}.jpg`.
3. Database record insertion: Inserts a row into the `alerts` table.

```
alert = Alert(
    job_id=job_id,
    alert_type="pedestrian",
    title="Crowd Threshold Exceeded",
    description=f"{person_count} persons detected inside ROI",
    frame_number=frame_number,
    image_path="media/alerts/{job_id}/pedestrian_{frame_number}.jpg",
)
db.add(alert)
db.commit()
```

6. API Specification for Developers

All endpoints require standard authentication dependencies (`require_operator` or `require_viewer`).

6.1 Video Upload with Pedestrian Config

Uploads a static video file and defines pipeline features and crowd threshold.

- **Method:** `POST`

- **Path:** /upload-video
- **Headers:** Authorization: Bearer <JWT_TOKEN> (Operator role required)
- **Form Data:**
 - file: UploadFile (Video file, e.g., .mp4, .avi)
 - analysis_config: string (JSON string) -> {"vehicle": true, "plate": true, "pedestrian": true}
 - pedestrian_threshold: integer (Default: 10)

Example Request (cURL):

```
curl -X POST "http://localhost:8000/upload-video" \  
-H "Authorization: Bearer <TOKEN>" \  
-F "file=@sample.mp4" \  
-F 'analysis_config={"vehicle": true, "plate": false, "pedestrian": true}' \  
-F "pedestrian_threshold=5"
```

Response (200 OK):

```
{  
  "job_id": "a1b2c3d4-e5f6-7890-abcd-1234567890ab",  
  "status": "uploaded"  
}
```

6.2 Live RTSP Camera Job Creation

Configures a live RTSP IP camera stream with pedestrian monitoring.

- **Method:** POST
- **Path:** /camera-job/create
- **Headers:** Authorization: Bearer <JWT_TOKEN> (Operator role required)
- **Request Body** (application/json):

```
{  
  "username": "admin",  
  "password": "secretpassword",  
  "ip_address": "192.168.1.100",  
  "path": "/h264",  
  "name": "Entrance-Camera-01",  
  "analysis_config": {  
    "vehicle": false,  
    "plate": false,  
    "pedestrian": true  
  },  
}
```

```
"pedestrian_threshold": 8
}
```

Response (200 OK):

```
{
  "job_id": "c9d8e7f6-a5b4-3210-9876-fedcba098765",
  "status": "uploaded",
  "job_type": "camera_stream"
}
```

6.3 Setting ROI & Triggering Background Pipeline

Sets ROI polygon coordinates and kicks off pipeline processing in a background worker thread.

- **Method:** POST
- **Path:** /job/set-roi-line (or /camera-job/{job_id}/start for streams)
- **Headers:** Authorization: Bearer <JWT_TOKEN> (Operator role required)
- **Request Body** (application/json):

```
{
  "job_id": "a1b2c3d4-e5f6-7890-abcd-1234567890ab",
  "roi_coords": [
    [100, 200],
    [500, 200],
    [500, 600],
    [100, 600]
  ],
  "line_coords": null,
  "line_distance_meters": null
}
```

Response (200 OK):

```
{
  "job_id": "a1b2c3d4-e5f6-7890-abcd-1234567890ab",
  "status": "pending",
  "message": "Processing started with ROI and line"
}
```

6.4 Fetching Results & Crowd Alerts

Retrieves all job data, detected plates, and crowd threshold violation alerts.

- **Method:** GET
- **Path:** /job/{job_id}/results
- **Headers:** Authorization: Bearer <JWT_TOKEN> (Viewer or Operator role required)

Response (200 OK):

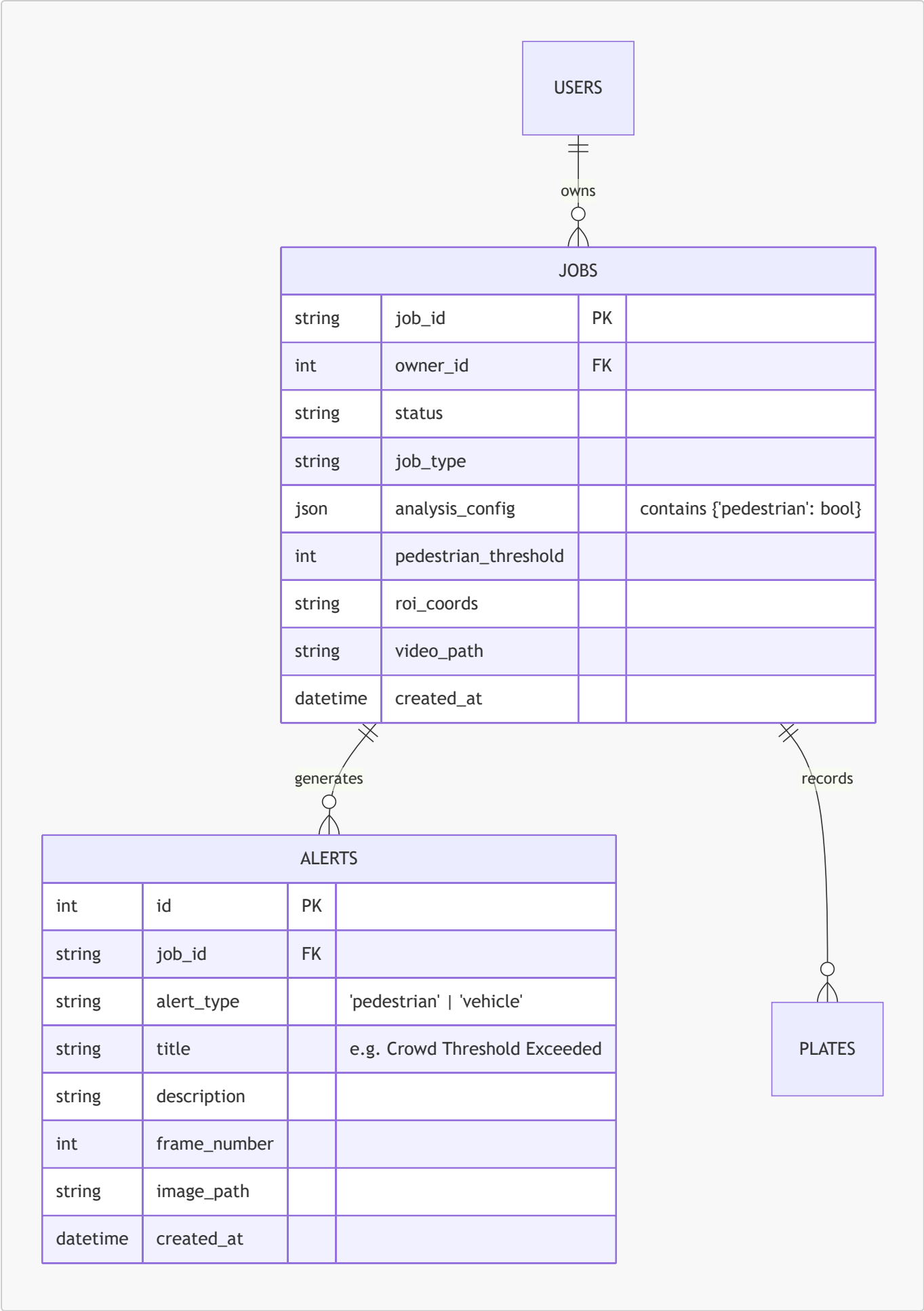
```
{
  "job_id": "a1b2c3d4-e5f6-7890-abcd-1234567890ab",
  "status": "completed",
  "processed_video": "media/processed/a1b2c3d4..._final.mp4",
  "total_plates": 0,
  "plates": [],
  "total_alerts": 2,
  "alerts": [
    {
      "id": 14,
      "alert_type": "pedestrian",
      "title": "Crowd Threshold Exceeded",
      "description": "12 persons detected inside ROI",
      "frame_number": 150,
      "image_path": "media/alerts/a1b2c3d4.../pedestrian_150.jpg",
      "created_at": "20-08-2026 16:45:10"
    },
    {
      "id": 15,
      "alert_type": "pedestrian",
      "title": "Crowd Threshold Exceeded",
      "description": "15 persons detected inside ROI",
      "frame_number": 450,
      "image_path": "media/alerts/a1b2c3d4.../pedestrian_450.jpg",
      "created_at": "20-08-2026 16:45:25"
    }
  ]
}
```

6.5 WebSocket Live Streaming

Streams live annotated JPEG frames directly to the frontend for camera streams.

- **Protocol:** WebSocket
- **Path:** /ws/camera-job/{job_id}/live?token=<JWT_ACCESS_TOKEN>
- **Payload Format:** Binary JPEG byte frames (image/jpeg)
- **Control Messages (JSON):**
 - Error: {"type": "error", "message": "Reason..."}
 - Completion: {"type": "done", "status": "completed"}

7. Database Schema & Entity Relations



8. Notification & Email Reporting

When a job completes processing:

1. `app/services/job_manager.py` spawns a background thread targeting `NotificationService.send_completion_email(job_id)`.
2. `app/email/notification.py` queries all alerts associated with the job and calculates pedestrian statistics:

```
pedestrian_alerts = sum(  
    1 for alert in alerts if alert.alert_type.lower() == "pedestrian"  
)
```

3. `app/email/email_template.py` formats an HTML report containing:
 - Total Pedestrian Crowd Alerts triggered.
 - Direct image attachments / links for snapshot evidence.
 - Job metadata and overall status.

9. Intern Onboarding & Debugging Guide

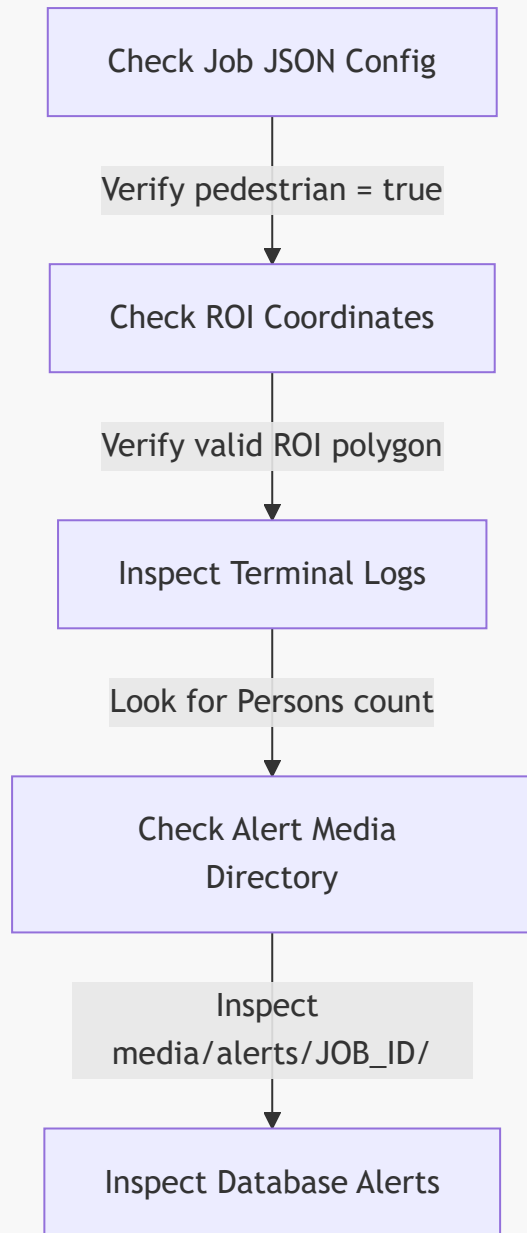
9.1 Local Environment Setup

1. **Verify Weights File:** Ensure `models/pedestrian.pt` is present in the repository root directory.
2. **Environment Variables:** Verify `.env` configuration:

```
DATABASE_URL=postgresql://user:pass@localhost:5432/anpr_db  
SECRET_KEY=your_jwt_secret_key
```

9.2 Tracing a Pedestrian Bug Step-by-Step

If pedestrian detection or alerting is failing in a job, follow this step-by-step diagnostic workflow:



1. **Check `analysis_config`:** Query the DB or check the upload API request to ensure `"pedestrian": true` was sent.
2. **Verify ROI Format:** Inspect `job.roi_coords`. It must be a valid JSON array of points `[[x1,y1], [x2,y2], ...]`.
3. **Terminal Output:** Check the backend console output while pipeline runs:

```
[INFO] Frame 120 | Vehicles: 2 | Persons: 14
```

4. **Inspect Alert Images:** Check `media/alerts/{job_id}/` to verify whether JPEGs are being rendered and saved to disk properly.
5. **Database Query:** Run SQL to inspect generated alerts:

```
SELECT id, alert_type, title, description, frame_number, image_path
FROM alerts
WHERE job_id = 'YOUR_JOB_ID' AND alert_type = 'pedestrian';
```
